

Tutorial and Practical work n° 2: Linked Lists

Level: 1st Year

Material: Algorithms and Dynamic Data Structures

Exercise 01: Create a linked list from input data

Develop a module to build a linear linked list from a set of user-provided data.

Exercise 02: Length of a linked list

Write a module to calculate the length of a linked list (number of nodes).

Exercise 03: Find element with the most occurrences

Write a module to search for the element that occurs the most in the linked list.

Exercise 04: Access element by value and by position

1. Write a module that returns the first node containing a given value **v** in the linked list.
2. Write a module to access the node at a specific position **k** in a linked linear list.

Exercise 05: Delete by value and by position

1. Create a module to delete the first node that contains a specified value **v** in a linked linear list.
2. Write a module to delete a node at a specified position **k** in a linked linear list.

Exercise 06: Insert by position

Write a module to insert a new node at a specific position **k** in a linked linear list.

Exercise 07: Merge two sorted lists

1. Write a module that merges two sorted linked linear lists, **I1** and **I2**, into one sorted list.
2. Write a module that merges two linked lists, **I1** and **I2**, into one without allocating additional memory.

Exercise 08: Split a list based on a criterion

Develop a module to split a linked linear list into two new lists based on a specific criterion (e.g., even/odd).

Exercise 09: Find the node with the minimum value

Implement a module to find the node with the smallest value in the linked linear list.

Exercise 10: Delete all nodes with a specific value

Write a module to delete all the nodes with a given value **v** from the linked linear list.

Exercise 11: Reverse the linked list

Implement a module that reverses the linked linear list.

Exercise 12: Remove duplicates from an unsorted linked list

Write a module to remove duplicate values from an unsorted linked list without using extra memory.

Exercise 13: Check if a linked list is a palindrome

Develop a function that checks if a linked linear list contains a palindrome, i.e. that reads the same forward and backward.

Exercise 14: Add a node to the end of a list

Implement a module that adds a node to the end of the linked linear list only if it is empty.

Tutorial and Practical work n° 2: Linked Lists

Level: 1st Year

Material: Algorithms and Dynamic Data Structures

Exercise 15: Selection sort

Implement the selection sort algorithm to sort the elements of a linked linear list.

Exercise 16: Reverse a linked list in groups of size K

Write a module to reverse the linked linear list in groups of size k.

Exercise 17: Rotate a linked list

Write a function to rotate the linked linear list (move nodes to the left or right in a circular fashion).

Exercise 18: Delete nodes with values below a threshold

Develop a function that deletes all the nodes from a linked linear list whose values are less than or equal to a given threshold.

Exercise 19: Names and dates

Given a linked linear list, that contains string names and dates:

1. Develop a module to implement and fill this list structure.
2. Develop a module to remove duplicate nodes (same names and dates).
3. Develop a module that returns the intersection of three lists.

Exercise 20: Prime numbers

Given an integer number $n \geq 5$:

1. Write a function based on linked linear lists that returns all the prime numbers less than or equal to n.
2. Write the main algorithm.

Example: For $n=5$, the **function** returns 2, 3 and 5.

Exercise 21: Union and intersection lists

Given two linked lists **l1** and **l2**:

1. Write a function **unionLists** that takes as input lists **l1** and **l2** and returns their **union**.
2. Write a function **intersectionLists** that takes as input lists **l1** and **l2** and returns **intersection**.

Exercise 22: Print list

Given a linked list, **dll**, of size **n**, where every node represents a linked sub-list and contains two pointers:

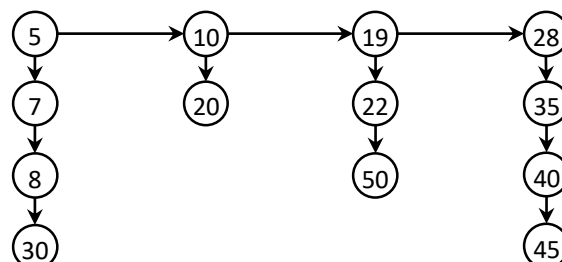
- **Next:** points to the next node.
- **Bottom:** points to a linked list where this node is the **head**.

Each of the linked sub-list is sorted in ascending order.

1. Propose the most suitable data structure for a node of such a linked list.
2. Write a module to print the linked list as it appears in the previous example.

Example:

Input:



Tutorial and Practical work n° 2: Linked Lists

Level: 1st Year

Material: Algorithms and Dynamic Data Structures

Output:

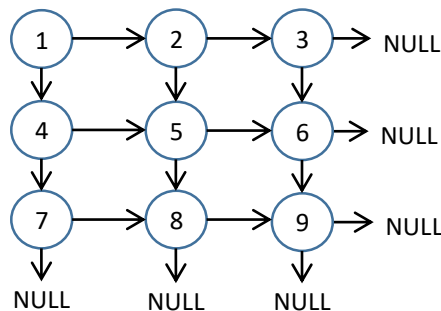
- 5 → 7 → 8 → 10 → 19 → 20 → 22 → 28 → 30 → 35 → 40 → 45 → 50

Exercise 23: Matrix to lists

Given a matrix **mat** of a size **n*n**:

- Propose the most suitable data structure.
- Write an algorithm that builds a **2D linked list L**, representing the matrix.

Example: mat = [[1 2 3], [4 5 6], [7 8 9]]



- Write the main algorithm.

Exercise 24: Multiplication of two numbers

Given two numbers **m** and **n** represented by two linked linear lists **L1** and **L2**:

- Propose the most suitable data structure.
- Write an algorithm that returns the result **L** representing the multiplication **mul** of both LLL **L1** and **L2**.
- Write the main algorithm.

Example:

- Input:** m=32 represented by **L1**: 3 ↔ 2 ↔ NULL; n=13 represented by **L2**: 1 ↔ 3 ↔ NULL
- Output:** mul=416 represented by **L**: 4 ↔ 1 ↔ 6 ↔ NULL

Exercise 25: Merge several lists of different sizes

Given **k sorted linked linear lists** of different sizes, their heads are stored in a given array **Tab**:

- Propose the most suitable data structure.
- Write an algorithm that merges them together while maintaining their sorted order. For example:
 - k=3, Tab: Head1 | Head2 | Head3
 - Head1: 1 → 3 → 5 → 7 → NULL
 - Head2: 2 → 4 → 6 → 8 → 10 → NULL
 - Head3: 0 → 9 → 11 → NULL
 - Result: 0 → 1 → 2 → 3 → 4 → 5 → 6 → 7 → 8 → 9 → 10 → 11 → NULL
- Write the main algorithm.

Exercise 26: Polynomial integration

A polynomial is defined as $P(x) = a_n x^n + a_{n-1} x^{n-1} + \dots + a_1 x + a_0$, where each term is represented by a coefficient and an exponent.

- Implement a linked linear list to store the polynomial.
- Develop the following modules:
 - Procedure setNext(p, q: *Term);
 - Procedure setExp(p: *Term; exp: integer);

Tutorial and Practical work n° 2: Linked Lists

Level: 1st Year

Material: Algorithms and Dynamic Data Structures

- Function `getCoef(p: *Term): real`;
 - Procedure `setCoef(p: *Term; coef: real)`;
 - Function `getNext(p: *Term): *Term`;
 - Function `getExp(p: *Term): integer`;
3. Write a function to insert a term while maintaining the order of exponents in descending order.
 4. Implement a function to display the polynomial in a readable format.
 5. Write a function to compute the indefinite integral of the polynomial. The result should be a new polynomial where each term follows the rule: $\int a_i x^i dx = \frac{a_i}{i+1} x^{i+1} + C$, where C is constant of integration.
 6. Write the main algorithm.

Example: Integral of $P(x) = 3x^3 + 2x + 5$ should be $\int P(x)dx = \frac{3}{4}x^4 + x^2 + 5x + C$, C is a constant of integration.

Exercise 27: Rotate a list by k position

A linked linear list contains integers:

1. Write a module to implement and fill this list data structure.
2. Write a module to rotate the list by k position.

Exercise 28: Palindrome

1. Write a module that checks if a linked list is a palindrome.
2. Using the previous module, write another module that returns the maximum palindrome of a linked list.

Exercise 29: Create a doubly linked list from input data

Implement a module to create a doubly linked list from user-provided data.

Exercise 30: Insert an element in a doubly linked list

Write a function to insert an element into a doubly linked list.

Exercise 31: Delete an element from a doubly linked list

Create a function to delete an element from a doubly linked list.

Exercise 32: Insert and delete nodes in a bidirectional linked list

1. Develop a module that inserts a node at any position in a doubly linked list.
2. Develop a module that deletes a node at any position in a bidirectional linked list.

Exercise 33: Remove duplicate nodes in unsorted doubly linked list

Given an **unsorted** doubly linked list **dll** containing **n** nodes:

1. Propose the most suitable data structure.
2. Write an algorithm that removes the **duplicate nodes** while **preserving** the order.
3. Write the main algorithm.

Example:

- **Input:** Doubly Linked List **dll**: $1 \leftrightarrow 2 \leftrightarrow 3 \leftrightarrow 2 \leftrightarrow 4$
- **Output:** Doubly Linked List **ddl**: $1 \leftrightarrow 2 \leftrightarrow 3 \leftrightarrow 4$

Exercise 34: Students marks

1. Write a module that takes the class semester marks for **n** students as input of a linked list where the students names and marks are added in sequence.
2. Develop the following modules:

Tutorial and Practical work n° 2: Linked Lists

Level: 1st Year

Material: Algorithms and Dynamic Data Structures

- Delete student without marks (null), then compute the new students number.
- Compute the average marks of the class.
- Compute the number and the rank of students who get marks greater than or equal to 10; then print their names.
- Print all students names that start with a given character **car**.
- Sort the list by student names in ascending order.
- Sort the list by marks in descending order.
- Add new student while preserving the order of the list.

Exercise 35: Add and delete nodes in a double linked list

1. Write a module to implement and fill a double linked list.
2. Write a module to add an element in the previous list:
 - At the beginning of the list.
 - At the end of the list.
 - At a specific position.
3. Write a module to remove an element in the previous list:
 - At the beginning of the list.
 - At the end of the list.
 - At a specific position.

Exercise 36: Circular linked list

1. Write a module to implement and fill a circular linked list.
2. Write a module to add an element in the previous list:
 - At the beginning of the list.
 - At the end of the list.
3. Write a module to remove an element in the previous list:
 - At the beginning of the list.
 - At the end of the list.

Exercise 37. Construct a circular linked list

Implement a module to create a circular linked list where the last node points back to the first node.

Exercise 38: Implement a circular doubly linked list

Write a module to create a circular doubly linked list where nodes point to both the next and previous nodes, and the list wraps around itself.